EidosSQL

εἶδος · the form, the thing seen

Master Guide — every feature, how it was built,
and why it was built that way

An interactive SQL visualizer for DSO 435 · Data Base Management Systems

Version 1.1 · July 2026

~5,500 lines of TypeScript · SQL engine written from scratch

Verified against PostgreSQL: 61/61 differential queries identical

> *In ancient Greek, **εἶδος** is the form — the shape by which a thing is known. In modern Greek, a **type** or kind. EidosSQL makes the form of a query visible: one step, one type, one table at a time.*

Contents

How to use this guide

Three reading paths, depending on why you opened it:

If you're…	Read
Preparing for an interview	Part 0 (pitch + stories), then Part III (trade-offs), Part IV (complexity), Appendix A (Q&A), Appendix B (vocabulary)
Returning to the code after months away	Part II (architecture, file map), Appendix C (the crown-jewel code), Part VI (workflows)
Explaining the project to someone	Part I (product tour), then §21's demo script

Parts I–II describe *what exists*. Part III is the one that matters most in an interview: it records the decisions, the alternatives that were rejected, and what each choice cost — the reasoning that a finished codebase silently erases.

Part 0 — Interview toolkit

0.1 The 30-second pitch

> *"EidosSQL is a teaching tool that shows what a SQL query does while it runs. You type a query and it replays it in the database's logical order — FROM, then JOIN, then WHERE, then GROUP BY, and only then SELECT — as a sequence of animated table states, so you watch rows get matched, filtered, grouped, and collapsed. To do that I had to write the SQL engine from scratch — tokenizer, parser, and an evaluator that snapshots itself after every clause — because off-the-shelf engines only hand you the final answer. It's differential-tested against PostgreSQL: 61 queries run through both my engine and a real Postgres in CI, diffed cell by cell."*

Three things that pitch is engineered to do: name a **real user problem** (the professor drawing tables on a whiteboard during office hours), state a **non-obvious technical constraint** (why an existing engine can't be used), and end on **evidence** (differential testing) rather than a claim.

0.2 The three-minute walkthrough

Problem. In DSO 435, students hit a wall at joins and grouping. During office hours the professor kept re-drawing the same thing on a whiteboard: here's the table, here's what the join does to it, here's what's left after the WHERE. It's the same explanation every time, it doesn't scale past one person, and it disappears when the whiteboard is erased.

Why existing tools don't solve it. A database gives you the final result. Query plans (`EXPLAIN`) show the physical strategy, not the data. Neither shows the *intermediate data* — which is exactly the thing students can't picture.

Constraint that shaped everything. To animate intermediate states, the evaluator has to pause after every logical clause and snapshot the working relation — and crucially, rows must keep a **stable identity** across snapshots so the UI can tell "this row survived" from "a different row is here now." No embeddable engine exposes that. So: write one.

What I built. ~5,500 lines of TypeScript. A tokenizer and a recursive-descent parser where every AST node carries a source span (that's what lets the editor highlight exactly the clause the current step belongs to), and an executor that evaluates the query for real while emitting a snapshot per stage. CTEs, subqueries, and UNION branches recurse as nested step groups. On top: a React UI where framer-motion FLIP animations turn the row-identity system into visible motion.

How I know it's right. A differential test: the same 61 queries run through my engine and a real PostgreSQL loaded with identical data, results diffed cell by cell. It found two bugs I'd never have caught by

eye. It runs in CI against a `postgres:16` service container on every push, alongside 62 unit tests that cover what the differential test structurally cannot see — the error messages and the *steps themselves*.

Where it went. Built as a TA tool for the professor who hired me; runs against the class database, any local Postgres, or uploaded CSVs.

0.3 Six stories worth telling

Each is STAR-shaped (situation → task → action → result) and each demonstrates a different competency. Pick to match the question.

① **The sampling limitation, and fixing the real cause.** *Competency: diagnosing the actual bottleneck instead of the symptom.* Connect-your-own-database initially sampled 300 rows per table, and I told myself that was a pedagogical choice — nobody learns from 7,000 animated rows. When the user pushed back ("there's really no way to work on a whole table like DBeaver does?"), I looked again and admitted the honest answer: sampling was compensating for an O(n×m) nested-loop join. Real databases don't scan; they hash. I added a join planner that flattens the ON clause into conjuncts, classifies each `=` by which side its columns come from, and uses clean splits as hash keys — leaving anything else as a residual per-pair test, with the nested loop as fallback for pure inequality joins. Full Parch & Posey (352 accounts × 6,912 orders) went from *refused* to a complete 8-step visualization in **38 ms**, results identical to `psql`. The lesson I'd state out loud: a limitation you've rationalized is still a limitation.

② **The differential test that caught what review couldn't.** *Competency: choosing a strong oracle.* Hand-writing expected values for SQL is both laborious and circular — you tend to encode the same misunderstanding into the test that's in the code. So instead of asserting values, I asserted *agreement*: load identical data into a scratch Postgres, run the same query through both, diff cell by cell (NULL-tokenized, float-tolerant, multiset comparison unless an ORDER BY makes order meaningful). It immediately caught two real bugs (§18.2) — one of them a SQL rule I simply didn't know. This is the single most reusable idea in the project: **when a reference implementation exists, test against it, not against your own expectations.**

③ **Correlation detection without static analysis.** *Competency: finding a simpler mechanism than the obvious one.* A correlated subquery (one referencing the outer row) must re-run per row; an uncorrelated one should be evaluated once and cached. The textbook approach is a static pass over the AST resolving every identifier against enclosing scopes — a lot of machinery, duplicated with the resolver I already had. Instead I let the existing name resolution answer the question at runtime: push the enclosing row context onto an outer-scope stack with a boolean tripwire; if column resolution ever falls through to that outer frame, the flag trips. Correlated ⇒ don't cache, re-run per row. Uncorrelated ⇒ cache. The flag also drives the user-facing narration ("this subquery depends on the outer row, so it re-runs for every one"), so one mechanism serves both correctness and pedagogy. ~15 lines instead of a resolver pass.

④ **Making the invisible visible (the row-identity system).** *Competency: designing a data model to serve a UX requirement*. The animation isn't decoration — it's the whole explanation. For the UI to show a row *moving* rather than disappearing and reappearing, the engine must guarantee identity across snapshots. Base rows are `alias:table:i`, joined rows compose as `left⋈right`, NULL-extended rows as `left⋈ø`, group rows as `g:<key>`. React keys off those ids, framer-motion FLIPs the layout difference, and the engine never contains a line of animation code. Clean separation: the engine promises identity; the UI renders motion.

⑤ **Timeline chips as a second information channel.** *Competency: hearing the real need behind a feature request*. The request was "shade the step buttons by table." The underlying need was that in a query with CTEs and subqueries, students lose track of *which table each step is even about*. So relations now carry provenance — the tables, CTEs, and subquery aliases they derive from — propagated through the whole pipeline (FROM sets it, JOIN concatenates, GROUP BY carries, set-ops merge). Steps tag their sources with stable color slots; a join chip renders half-and-half in its two sides' colors. The payoff shows up on the capstone query: the HAVING subquery's chip wears the CTE's color, so you can *see* it reads the CTE without re-reading the SQL. Unit tests now pin that behavior.

⑥ **Errors as curriculum.** *Competency: product thinking in an unglamorous place.* 64 error sites, 37 carrying a teaching hint. Using a SELECT alias in WHERE doesn't say "column does not exist" — it says the alias exists but WHERE runs *before* SELECT names it, and suggests repeating the expression or wrapping the query. Errors carry exact source spans so the editor underlines the offending token. These are the app's second curriculum, and they're the part most likely to rot silently, so 18 unit tests pin each message, hint, and span.

0.4 Question → section map

If they ask…	Go to
"Walk me through the project"	§0.2
"What was the hardest part?"	§0.3 ① or ③; §11 (join planner)
"Why not use an existing library?"	§III.1
"How did you test it?"	§0.3 ②, §18, Appendix A-9
"What's the time complexity?"	Part IV
"Tell me about a bug you found"	§18.2 (three war stories)
"What would you do differently?"	§III (every "revisit" line), §22
"How does it scale?"	§IV.2, §19 (ceilings)
"Tell me about a design trade-off"	Part III — pick any
"How do you handle X in the browser?"	§14 (bridge), §IV.3 (work budget)
"What did you learn?"	§22

Part I — Product tour

1. Layout

A single-page app with a fixed top bar (brand, **Database** picker, **Examples** picker, theme and presentation toggles) and two panes:

- **Left pane** — SQL editor, ▶ Visualize button (also ⌘/Ctrl+Enter), error panel, and a collapsible schema reference for the active database.
- **Stage** (right) — the step player: step header, animated table, playback controls, and the step timeline.

Below ~920 px the panes stack vertically (usable on a tablet; desktop is the primary target). Light and dark themes are both first-class.

2. The editor

- **Syntax highlighting** — keywords, strings, numbers, functions, comments, identifiers each get a token color (text-contrast-safe in both themes).
- **Clause spotlight** — while stepping through a visualization, the exact SQL span responsible for the current step is highlighted and auto-scrolled into view. This is the thread that ties "what the query says" to "what the database is doing."
- **Error underline** — parse/runtime errors wavy-underline the exact offending span; the panel below shows the message with a line number and, for classic student mistakes, a 💡 teaching hint (see §13).
- Tab inserts spaces; ⌘/Ctrl+Enter runs. If you edit after running, an "edited — run again to update" note appears and the stale highlight is suppressed.

3. The step player

Pressing **Visualize** replays the query in the database's *logical* clause order — not the order you wrote it:

FROM → JOIN(s) → WHERE → GROUP BY → collapse → HAVING → window functions → SELECT → DISTINCT → set operations → ORDER BY → LIMIT/OFFSET → RESULT

Each step shows:

- **Path badges** — nesting context (`WITH acct_orders`, `UNION — branch 2`, `Subquery in HAVING`), so CTEs, subqueries, and set-op branches read as stories-within-the-story.
- **Title** — the clause, verbatim (`WHERE total > 500`).

- **Description** — plain English with *real row counts* ("14 of 59 rows pass; 45 are removed").
- 💡 **Insight** — an optional teaching callout (why WHERE can't see aliases; what LEFT JOIN promises; what LIMIT without ORDER BY doesn't).
- **The table** — the animated state of the data at that moment.

Controls: ← / → keys, Prev/Next buttons, Play with 0.5×–2× speed, clickable timeline chips (indented `›` for nested steps), and a step counter. Row counts in each description tick up in a quick count-up animation (disabled for reduced-motion users), pulling the eye to the step's payload.

Timeline chips are color-coded by source. Every chip is tinted by the table(s), CTE(s), or subqueries its step is working on — assigned stable colors in order of first appearance and listed in a legend above the timeline. A JOIN chip is shaded **half-and-half** with its two sides' colors; steps inside a CTE wear that CTE's ingredients' colors; the "CTE ready" chip introduces the CTE's own color, which then reappears wherever the main query uses it — including inside a HAVING subquery that reads it. The goal: students can glance at the chip row and know *which table each clause is about* without re-reading the query, precisely when CTEs and subqueries make that hardest. A Greek-key (meander) rule underlines the timeline — one of exactly two Greek identity touches in the UI (the other is the wordmark's column-capital "E").

4. The animated table — reading the visuals

Rows keep a **stable identity** across steps, so the animation between two steps *is* the explanation:

Visual	Meaning
✓ green row	passed this step's test (WHERE/HAVING/LIMIT keep)
× red row, dimmed	failed; it fades out on the next step
+ blue row	appeared at this step (NULL-filled outer-join row, group row)
Colored left edge + wash	group / partition / join-provenance membership (8-color validated palette)
Blue-underlined column	column computed at this step (aggregate, window value, expression)
Gray pill on a row	per-row note ("no match — removed by INNER JOIN", "duplicate removed by UNION")
NULL in italics	a real SQL NULL, styled distinctly from text
Rows sliding	ORDER BY reordering (FLIP animation — no row appears or disappears)

Steps display the first 100 rows (with an "… N more rows" footer); the final RESULT view shows up to 1,000. Computation always covers *all* loaded rows — the caps are presentation only.

5. Databases

- **Embedded: Parch & Posey** — the class teaching database, trimmed to a referentially-intact 12-account slice (30 orders, 32 web events, all 7 regions, 10 reps) so every row fits on screen. The trim deliberately keeps teaching edge cases: **Mattel** has no orders (LEFT JOIN lessons), **International** has a rep but no accounts, **South/North** have nothing.
- **Connect your own Postgres** (Database → "➕ Connect your own Postgres…") — point the app at any database on your machine (`postgres://localhost:5432/northwind` , or just `northwind`). Choose a sample size from 100 rows up to **all rows** (50,000/table hard cap). If any table is sampled, the schema panel labels it ("300 of 6,912 rows (sample)") and a persistent amber banner warns that results on a sample can differ from the full database. With "all rows," results are exact.
- **Load CSV files** (Database → "📄 Load CSV files…") — each file becomes a table named after the file; the first row supplies column names and types are inferred per column (integer, numeric, date, timestamp, boolean, text; blank cells become NULL). Everything parses in the browser — nothing is uploaded anywhere. This lets an instructor hand out any dataset as plain CSVs with zero database setup.

5a. Presentation mode, themes, and printing

- 🎬 **Present** (top bar, Esc to exit) is the projector view: the editor pane disappears, a read-only syntax-highlighted copy of the query docks above the stage — keeping the clause spotlight visible — and type sizes step up across the board.
- ☀️ /🌙 **theme toggle** — the manual choice persists and overrides the OS setting (projectors want light mode regardless of the presenter's laptop).
- **Printing** any step produces a clean handout: light-forced colors, the full query with highlighting, the step narration, and the table laid out for paper with the app chrome stripped — homework material for free.

6. Examples

21 curated queries in the Examples menu, ordered like the semester:

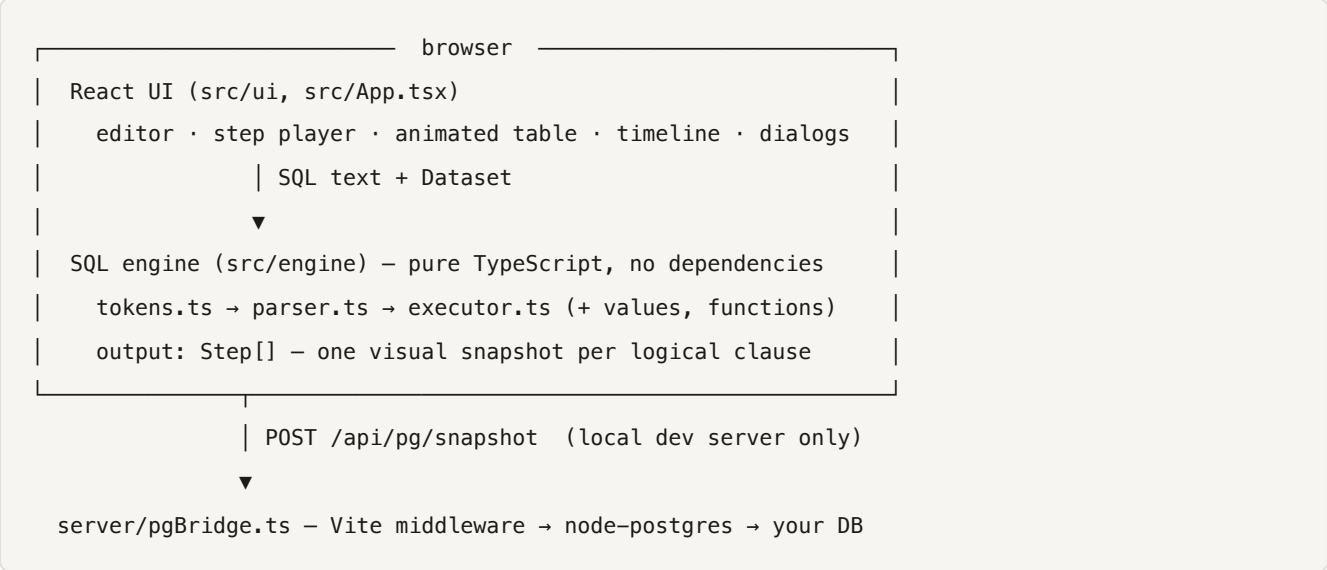
1. **Foundations** — SELECT, WHERE, ORDER BY + LIMIT, CASE
2. **Joins** — JOIN, chained joins, LEFT JOIN, LEFT JOIN + IS NULL
3. **Aggregation** — GROUP BY, HAVING
4. **Subqueries & CTEs** — IN, scalar subquery, CTE, UNION
5. **Window functions** — running total, RANK per region, LAG, NTILE

6. **Putting it together** — three capstones at end-of-course difficulty: UNION + CTE + CASE · LEFT JOIN + HAVING-with-subquery · LAG inside a CTE

Selecting an example loads the query, switches to the right database, and runs it immediately. Every example is exercised by the smoke test, so a broken example fails CI.

Part II — How it's built

7. Architecture at a glance



Three data sources (embedded dataset, live Postgres snapshot, uploaded CSVs) all normalize to one `Dataset` shape, so the engine has exactly one input format and never knows where the data came from.

The engine is written from scratch (no sql.js, no parser library) for one reason: **off-the-shelf engines only give you the final answer.** To animate intermediate states the evaluator itself must pause after every logical clause, snapshot the working relation, and keep row identities stable. That requirement shaped everything below — see §III.1 for the full trade-off.

File map (line counts as of v1.1):

```
src/engine/tokens.ts       148   tokenizer + SqlError (positions, hints)
src/engine/ast.ts          215   AST types; every node carries a source span
src/engine/parser.ts       878   recursive-descent parser
src/engine/values.ts       109   value semantics: NULL logic, dates, intervals
src/engine/functions.ts    319   35 scalar functions + aggregates
src/engine/executor.ts   2,011   the evaluator/step-emitter (the heart)
src/engine/steps.ts         72   Step/VizTable model consumed by the UI
src/data/datasets.ts       152   embedded Parch & Posey slice
src/data/remote.ts          53   client for the Postgres bridge
src/data/csv.ts            149   CSV parser + type inference → Dataset
src/ui/*                 1,177   Editor, StepTable, Timeline, SchemaPanel,
                                 ConnectPanel, CsvPanel, SqlView, AnimatedDesc,
                                 highlight.ts, examples.ts, App.tsx
server/pgBridge.ts         161   local-Postgres bridge (Vite middleware)
scripts/smoke.ts            62   engine smoke test (26 cases)
scripts/verify.ts          222   engine-vs-Postgres differential test (61 cases)
tests/*.test.ts            470   62 Vitest unit tests
src/styles.css           1,090   design tokens, layout, print & present modes
```

8. Value semantics (`values.ts`)

- Values are plain JS: `number | string | boolean | null`, plus an `Interval` object for timestamp arithmetic.
- **Dates/timestamps are ISO strings** (`2016-12-24 05:53:13`). Deliberate: they display exactly as Postgres prints them, compare correctly as plain strings (ISO-8601 is lexicographically ordered), and never drift through timezones. Date math parses them as UTC on demand. (Trade-off: §III.6.)
- **Three-valued logic** is enforced at the comparison layer: `compareValues` returns `null` when either side is NULL; `truthy()` implements "NULL and false both reject the row"; AND/OR/NOT propagate unknowns Postgres-style. This is why `NULL = NULL` correctly filters rows out and `x IN (…, NULL)` returns NULL rather than false — the mechanism behind the classic `NOT IN` trap.
- **Interval decomposition** mirrors Postgres: a timestamp difference is days + hh:mm:ss, so `EXTRACT(hour FROM (a - b))` gives the *hour component*, not total hours — matching real query results.
- `groupKey(values)` builds the canonical string key used everywhere equality-with-NULLs matters (GROUP BY, DISTINCT, set ops, hash joins, IN sets) — with SQL's twist that NULLs *are* equal for grouping. One function, five call sites, no divergence.

- **Integer division**: Postgres computes `7 / 2 = 3`. The engine reproduces this via a lightweight static type pass — dataset columns carry an `isInt` flag, and `isIntExpr()` propagates int-ness through literals, arithmetic, `count/sum/min/max`, CASE, and casts (memoized per AST node). Division truncates only when both operands are integer-typed. A classic gotcha the course teaches, faithfully reproduced.

9. Tokenizer and parser

`tokens.ts` produces tokens with byte-exact source positions (60 keywords, numbers, strings with `''` escapes, quoted identifiers, operators, both comment styles). The same tokenizer drives editor syntax highlighting, so the editor and engine can never disagree about what a token is.

`parser.ts` is a hand-written **recursive-descent** parser — one function per grammar production, precedence encoded in the call chain (`parseOr → parseAnd → parseNot → parseComparison → parseAdditive → parseMultiplicative → parseUnary → parsePostfix → parsePrimary`). Coverage:

- SELECT [DISTINCT] with expressions, aliases (`AS` or bare), `*` / `t.*`
- FROM with table aliases, subqueries-as-tables, and every join type (INNER/LEFT/RIGHT/FULL [OUTER]/CROSS, comma joins, ON and USING)
- WHERE · GROUP BY (expressions, ordinals, select aliases) · HAVING
- ORDER BY (expressions, ordinals, output names, ASC/DESC, NULLS FIRST/LAST) · LIMIT/OFFSET
- WITH (multiple CTEs, visible to later CTEs and the body)
- UNION [ALL] / INTERSECT / EXCEPT with correct precedence (INTERSECT binds tighter) and parenthesized branches that may contain their own WITH/ORDER
- Full expression grammar: OR→AND→NOT→comparisons (=, <>, <, <=, >, >=, IS [NOT] NULL/TRUE/FALSE, [NOT] IN list/subquery, BETWEEN, LIKE/ILIKE) →additive (+ − ||)→multiplicative→unary→ `::type` casts→primary
- CASE (searched and operand forms), CAST(x AS type), EXTRACT(field FROM x), INTERVAL '...', DATE/TIMESTAMP literals, EXISTS(...), scalar subqueries, function calls with DISTINCT args and OVER (PARTITION BY ... ORDER BY ... [ROWS frames])

Two design choices matter most:

1. **Every AST node records `{start, end}`.** That single invariant powers three separate features: the editor's clause spotlight, error underlines, and each step's `span`. Cheap to maintain while parsing, impossible to retrofit.
2. **Errors are written for students.** The parser distinguishes "syntax error" from "the mistake a student actually makes": a LEFT JOIN missing its ON explains cross-join blowup; a FROM-subquery

without an alias explains *why* SQL needs the name; keywords used as names suggest double quotes.

10. The executor — evaluation as storytelling

`executor.ts` evaluates the query for real while recording a `Step` after every logical stage. The `Step` model (`steps.ts`):

```
{ phase, chip, title, desc, insight?, span?, path[], sources[],  // narration
  table: { columns[], rows[], truncated? } }                     // the state
```

Stable row identity is the core trick. Base rows are `alias:table:i`; a joined row is `left⋈right`; a NULL-extended row is `left⋈∅`; a group row is `g:<key>`; set-op branches are prefixed `a:` / `b:`. Because ids persist across steps, the UI's FLIP layout animation renders "this row moved / survived / vanished" without the engine knowing anything about animation.

Source tracking works the same way for tables: every relation carries `sources` — the display names of the base tables, CTEs, and subquery aliases it derives from. FROM sets it, JOIN concatenates both sides, grouping/projection/set-ops carry or merge it, and each emitted step tags its sources with a stable color slot (assigned in order of first appearance). The timeline renders those tags as chip tints — one wash, a half-and-half join split, or stripes — plus the legend.

Pipeline details, in order:

- **FROM** resolves against the environment (dataset tables + CTEs), instantiates fresh column ids with the alias as `source` (that's the small gray label over each column), and errors on duplicate aliases with a self-join hint. Subqueries in FROM run first as a nested step group.
- **JOIN** — see §11.
- **WHERE** evaluates per row (aggregate/window usage rejected with hints), snapshots every row as kept/dropped, then keeps the survivors.
- **GROUP BY** resolves keys three ways (expression, ordinal, select alias — matching Postgres), builds groups in order of first appearance, and emits *two* steps: rows recolored & sorted into contiguous groups, then the collapse into one row per group. The grouped relation's columns are the keys plus every aggregate the rest of the query needs — collected up front by scanning SELECT, HAVING, and ORDER BY (deduped by normalized text; labeled by alias when a select item matches). Aggregates over an empty input with no GROUP BY still produce one row (`count(*)` = 0) — a real SQL quirk the differential test caught (§18.2).
- **Group-context evaluation**: after collapse, expressions are evaluated against a group context where (a) anything textually matching a GROUP BY key returns the key value, (b) a column reference resolving to a key column works under any alias spelling, (c) aggregate calls compute (and cache) over the group's member rows, and (d) any other bare column throws the canonical teaching error

("must appear in GROUP BY or be used in an aggregate — SQL no longer knows *which* row's value to show").

- **HAVING** is WHERE for group rows — same kept/dropped visual, plus the insight that explains the WHERE/HAVING division of labor.
- **Window functions** run after HAVING, over whatever rows remain (including grouped rows — `rank() over (order by sum(x))` works). Implementation: partition by `PARTITION BY` values → sort within partitions (stable, NULLS-aware) → compute per function. Supported: `row_number, rank, dense_rank, ntile, lag, lead, first_value, last_value` and any aggregate with OVER. Frames follow Postgres defaults — with an ORDER BY the frame is "start through current row *including peers*," which is why running totals tie on equal keys — plus explicit `ROWS BETWEEN n PRECEDING AND CURRENT ROW` frames for moving averages. The step shows partitions striped in color and the computed columns appearing; afterwards the relation keeps the window order (like Postgres output, and it makes LAG values visually traceable).
- **SELECT** happens *late*, as in real SQL: stars expand (excluding internal window columns), carried columns keep their ids (so the UI shows untouched columns persisting), computed columns get fresh ids and a "new" highlight. Labels: alias > column name > shortened expression text.
- **DISTINCT** dedupes whole output rows via `groupKey`, marking duplicates before removing them.
- **Set operations** run each branch as its own step group, enforce equal column counts (with a teaching message), stack rows with branch coloring, and show duplicate elimination (UNION), membership testing (INTERSECT), or subtraction (EXCEPT) row by row.
- **ORDER BY** resolves each key as output ordinal → unique output name → arbitrary expression evaluated against the *pre-projection* row (kept alongside by row id) — mirroring Postgres's ability to sort by non-selected columns, including its restriction that with DISTINCT the sort keys must be in the select list (enforced, with the explanation). Sorting is stable, multi-key, DESC-aware, with Postgres NULL defaults.
- **LIMIT/OFFSET** marks the cut rows before dropping them, and warns when there's no ORDER BY ("LIMIT keeps an *arbitrary* N rows").
- **RESULT** — the clean final table.

11. Joins: planner, hash path, and honesty about non-matches

The ON expression is flattened into AND-conjuncts. Each `=` conjunct is classified by which side its columns come from; clean left/right splits become **equi-join keys**, everything else stays a per-pair *residual* test.

- **Hash path** (any equi-keys): build a hash map of right-side rows keyed by `groupKey(keys)` (rows with NULL keys are excluded — NULL never equals anything), then probe from each left row and

apply residuals to the candidates. Bucket insertion order preserves table order, so output is identical to the nested-loop result. This is what makes full-size tables join in milliseconds — see §IV.2.

- **Nested-loop path** (no equality at all — inequality joins, cross joins): every pair is tested, guarded at 1.5M pairs with a hint to add an ON equality or pre-filter.
- **Outer-join semantics**: unmatched left rows appear as NULL-filled "+" rows (LEFT/FULL) or as × ghost rows annotated "no match — removed by INNER JOIN" (the single most clarifying visual in the app: inner vs. outer join is *shown*, not told). RIGHT/FULL append unmatched right rows. Matched rows are tinted by left-row provenance so students can see one account "fanning out" across its orders.
- `USING (col)` is rewritten to explicit equalities. (Deviation: Postgres merges the USING column into one; EidosSQL keeps both sides' columns.)
- Guards: 500k result rows max; a global **work meter** (30M evaluation ticks) aborts any runaway query with a teaching hint instead of freezing the tab.

12. Subqueries, CTEs, and correlation

- **CTEs** execute first as nested step groups, then register as tables in a scoped environment (visible to later CTEs and the body — including inside subqueries, which is how `(select avg(n) from some_cte)` works).
- **Scalar / IN / EXISTS subqueries** evaluate lazily at expression time. Uncorrelated ones are cached after one evaluation and — key pedagogy — their steps are emitted *inline, right before the clause that uses them*, ending with a "Subquery result: 2.125" step so students see the value get plugged in.
- **Correlation detection** needs no static analysis: when a subquery evaluates, the enclosing row context sits on an outer-scope stack with a tripwire flag. If column resolution falls through to the outer scope, the flag trips → the subquery is correlated → it isn't cached, it re-runs per row (silently after the first, whose steps are shown as the illustrative case), and the wrap-up step explains exactly that. Code in Appendix C-2; the reasoning in §0.3 ③.
- `IN (subquery)` membership uses a hashed set (with SQL's NULL-in-list semantics preserved), so it stays linear on full tables.
- Scalar subqueries enforce Postgres's contract with explanations: one column ("used as a single value"), at most one row ("use IN instead of =" when many rows come back), NULL when empty.

13. The teaching-error catalog

Errors are the app's second curriculum: **64 error sites, 37 with a teaching hint.** The notable ones:

Mistake	What EidosSQL says
SELECT alias used in WHERE/HAVING	"WHERE runs *before* SELECT names its outputs… repeat the expression, or filter a subquery/CTE"
Aggregate in WHERE	"WHERE filters rows before grouping — use HAVING"
Bare column with GROUP BY	"SQL no longer knows *which* single value to show — group by it or aggregate it"
Window function in WHERE/HAVING	"computed near the end — compute it in a CTE, then filter"
`"Walmart"` in double quotes	"double quotes name a *column*; use single quotes for text"
LEFT JOIN without ON	"without ON, every row pairs with every row"
FROM-subquery without alias	why the name is required, with the fix
Scalar subquery returns N rows	"comparing against many values? use IN"
Nested aggregates	"compute the inner aggregate in a CTE first"
Ambiguous column	lists the candidate tables, shows the qualified form
Unknown column/table/function	lists what *is* available
Division by zero	suggests `NULLIF(denominator, 0)`
UNION column-count mismatch	"set operations stack results vertically…"
WHERE after GROUP BY	word-order fix with the WHERE/HAVING rule
DISTINCT + ORDER BY on unselected column	why merging duplicates hides the column

14. The Postgres bridge (`server/pgBridge.ts`)

Browsers can't speak the Postgres wire protocol, so "connect your own database" is a 161-line middleware living *inside the student's own Vite dev server* (`configureServer` / `configurePreviewServer`):

- One endpoint: `POST /api/pg/snapshot {conn, limit}` → database name + every public-schema base table (max 40) with columns, total row count, and up to `limit` rows (hard cap 50,000; `limit ≤ 0` means "all").
- **Safety**: the session is forced `READ ONLY` ; only catalog queries and `SELECT … LIMIT` are ever issued; identifiers are quote-escaped.
- Type mapping: Postgres `data_type` → the engine's `integer/numeric/text/timestamp/date/boolean` ; `pg` type parsers are overridden so numerics arrive

as numbers and dates/timestamps as strings, then normalized to the engine's canonical formats (strip `T`, ms, tz).

- **Privacy**: connection strings never persist server-side and never leave the machine; a bare name like `northwind` expands to `postgres://localhost:5432/…`.
- On a static deployment the endpoint doesn't exist; the client detects the 404/network failure and explains that this feature needs `npm run dev`. (Graceful degradation was a deliberate requirement — §III.8.)

15. UI implementation notes

- **Editor** = a transparent `<textarea>` stacked on a highlighted `<pre>` with synced scroll — the classic overlay technique, but the highlight layer is built from the *engine's own tokenizer*, with span-splitting to overlay the active-clause and error ranges. Extracted to `src/ui/highlight.ts` so the editor, presentation mode, and print all share one implementation.
- **StepTable** renders `motion.tr` rows keyed by stable row id inside `AnimatePresence`: exits fade, survivors FLIP into place, entries fade in. Above 150 rows (the 1,000-row result view) it switches to static `<tr>`s — animating a thousand rows would burn CPU for zero pedagogy. Numeric cells right-align with `tabular-nums`.
- **Timeline** chips auto-scroll the active chip into view; nesting depth renders as `›` prefixes and dashed borders; source colors render as a single wash, a half-and-half linear-gradient, or up to 4 stripes.
- **App state** is deliberately boring React `useState`: the run is an immutable `Step[]`, so Prev/Next/Play/scrub are just an index change — stepping backwards is free and needs no inverse operations (§III.10).
- Playback pace is 2.4 s/step ÷ speed; keyboard arrows are ignored while typing in inputs; `MotionConfig reducedMotion="user"` plus a CSS reduced-motion block respect accessibility settings.
- **Presentation mode** is one boolean and a CSS class: `.presenting` hides the editor pane, reveals the docked `SqlView`, and scales typography. Esc exits. **Print** reuses the same `SqlView` via `@media print`, which also forces light tokens and unclips the table.
- **Theme** is a `data-theme` attribute on `<html>`: dark tokens apply under `prefers-color-scheme: dark` *unless* the user chose light, and under an explicit dark choice regardless of the OS. Persisted in localStorage.
- **Count-up descriptions** split the text on number tokens and animate each integer 0→value over ~500 ms (cubic ease-out, `requestAnimationFrame`), snapping to the exact original formatting at the end; decimals and reduced-motion users render statically.
- **CSV ingestion** (`src/data/csv.ts`) is a hand-rolled RFC-4180 parser (quoted fields, `""` escapes, CRLF, embedded newlines) plus per-column type inference by regex consensus over non-empty

values; conversion mirrors the embedded dataset's conventions so the engine treats all three sources identically.

16. Design system

Tokens follow a validated reference palette (categorical slots ordered to maximize worst-case color-vision-deficiency separation — verified with a palette validator in both light and dark, against each theme's actual surface):

- **Status colors** (kept/dropped/new) are semantically reserved and always paired with a glyph (✓ × +) and often a note — color is never the only carrier of meaning.
- **Group/partition colors** are ~10%-opacity washes plus a 3px left edge; the actual key values are always visible as text in the row.
- Chrome follows the token sheet (`--page/--surface/--ink/--grid/…`) with a full dark variant; code tokens are darkened/lightened per theme for text-grade contrast.
- System font stack; monospace reserved for SQL, identifiers, and the step title (which quotes the query).

17. Datasets

`src/data/datasets.ts` is generated from the live class database: 12 well-known accounts chosen to span four regions and nine reps, each with its first/middle/last orders and web events so time-series examples work, plus intact edge cases (§5). Rows are stored as compact JSON arrays with typed column definitions — the `integer` markers are what power Postgres-style integer division (§8).

Part III — Design decisions & trade-offs

The section an interviewer will spend the most time on. Each entry states the problem, the options, the choice, what it cost, and what I'd revisit.

III.1 Build the SQL engine, or embed one?

Options. (a) sql.js / SQLite compiled to WebAssembly. (b) A parser library (node-sql-parser) plus my own evaluator. (c) Everything from scratch.

Chose (c). The product requirement isn't "run SQL" — it's "show the intermediate state after every logical clause, with row identities stable across snapshots so motion is meaningful." Embedded engines expose results, not the evaluator's internal states; there's no hook that says "you've finished the WHERE clause, here's the working set." I'd have ended up re-running truncated variants of the query and *inferring* intermediate states — fragile and often wrong (you can't derive pre-GROUP BY rows from a grouped result).

Cost. ~3,800 lines of engine, and a permanent correctness burden: every SQL rule I get subtly wrong is a wrong lesson. That cost is what justifies the differential test — the two are a package deal.

What I'd revisit. Nothing about the decision; it was forced by the requirement. But I'd write the differential harness *first* next time (§22), because it turned out to be the thing that made the engine trustworthy, and building it earlier would have caught bugs sooner.

III.2 Snapshot-per-clause vs. replaying prefixes

Options. (a) Emit a snapshot inline as evaluation proceeds. (b) Re-run the query N times, each truncated to the first k clauses.

Chose (a). Prefix-replay is tempting because it needs no engine changes, but it's O(clauses) full evaluations, it can't represent steps that aren't valid queries (a "collapse" step, a NULL-extended join row), and it can't preserve identity across runs — the very thing the animation depends on.

Cost. The executor carries presentation concerns (`pushStep` calls interleaved with evaluation), so it isn't a pure evaluator. Roughly a third of `executor.ts` is narration: building descriptions, marking row statuses, tagging sources.

What I'd revisit. Separate the two with an event-emitter: the evaluator emits typed events (`joinMatched`, `rowFiltered`) and a separate "narrator" module turns events into `Step`s. Same output, better testability, and the engine would become reusable for non-visual purposes.

III.3 Stable row identities vs. positional diffing

Options. (a) Give every row a stable id the engine guarantees across snapshots. (b) Let the UI diff consecutive tables heuristically (by index or by content hash).

Chose (a). Positional diffing breaks exactly where the teaching value is: after an ORDER BY every index changes, so a heuristic reports "all rows replaced" instead of "the rows re-sorted." Content hashing fails on duplicate rows (which the DISTINCT lesson is *about*).

Cost. Every operator must thread ids through. Composite schemes (`left⋈right`) make ids long on multi-way joins; they're opaque strings, so debugging by eye is harder.

What I'd revisit. Interned integer ids with a side table, purely for memory on large joins. Not urgent — string ids have never been the bottleneck.

III.4 Sampling, then hash joins

Original decision. Connect-your-own-Postgres sampled 300 rows/table, justified as pedagogy ("nobody learns from 7,000 rows") and enforced by a 1.5M-pair join guard.

The flaw. That reasoning conflated *display* with *computation*. Displaying 100 rows is pedagogy; computing over only 300 is a wrong answer. The real constraint was an O(n×m) nested-loop join.

Fix. A join planner with a hash path (§11), plus a hashed set for `IN (subquery)`, plus a global work meter replacing the crude pair guard. Then the sample cap could rise to "all rows" (50k/table) because full-size joins became linear. Verified: full Parch & Posey join+group+sort in **38 ms**, identical to `psql`.

Cost. The planner adds real complexity — conjunct flattening, side classification, residual predicates, NULL-key exclusion — and two code paths to keep in agreement. Mitigated by differential tests that exercise both (`join with residual condition`, `non-equi join`).

Lesson worth stating in an interview. I had rationalized a limitation as a feature. The user's question ("is there really no way…?") was the prompt to re-examine it, and the honest answer was that my algorithm — not the pedagogy — was the constraint.

III.5 Runtime tripwire vs. static analysis for correlation

Options. (a) A static pass resolving identifiers against enclosing scopes before evaluation. (b) Detect at runtime by observing whether name resolution reaches an outer frame.

Chose (b) — see §0.3 ③. Reuses the resolver that must exist anyway; ~15 lines; and the same signal drives the narration.

Cost. The answer arrives *after* the first evaluation, so the first run of an uncorrelated subquery isn't cached in advance (harmless — it's cached immediately after). And correlation is discovered per call site rather

than proven for the query as a whole, so I can't use it for query-level optimization decisions the way a real planner would.

What I'd revisit. Nothing at this scale. A production optimizer needs the static pass because it must decide *before* execution.

III.6 ISO strings for dates instead of `Date` objects

Options. (a) JS `Date`. (b) A custom temporal class. (c) ISO-8601 strings, parsed to UTC on demand.

Chose (c). Three properties matter: they render exactly as Postgres prints them (no formatting layer), they compare correctly with plain string comparison (ISO-8601 is lexicographically ordered), and they can't drift through local timezones — a `Date` silently re-interprets in the browser's zone, which would make results differ between the student's laptop and the differential test.

Cost. Date arithmetic parses on demand (cheap at these sizes, wasteful at scale), and correctness depends on inputs being normalized at the edge — so the CSV parser and the Postgres bridge both normalize into the same format. That's a real invariant maintained in three places.

What I'd revisit. If the engine ever handled timezone-aware timestamps, this breaks and needs a proper temporal type. For a teaching subset it's the right simplification — and knowing *why* it's a simplification is the point.

III.7 Differential testing vs. hand-written expectations

Options. (a) Assert expected result sets by hand. (b) Assert *agreement* with a reference implementation.

Chose (b) as the primary oracle, with (a) as a supplement. Hand-written expectations for SQL are laborious and circular — the same misunderstanding that produced a bug tends to produce the "expected" value. Postgres is the ground truth the app claims to imitate, so imitation is exactly what should be asserted.

Cost. The deep suite needs a live Postgres, so it can't run everywhere — solved in CI with a service container, and locally with `createdb`. That's why the DB-free Vitest layer exists too: `npm test` must be meaningful on any machine.

Limits worth naming. A differential test only validates *final results of valid queries*. It cannot see error messages, and it cannot see the steps — which are the actual product. That gap is precisely what the unit suites cover (§18.1), and knowing where your strongest test is blind is the point.

III.8 Dev-server bridge vs. a hosted backend

Options. (a) Host a backend that proxies to student databases. (b) Ship a bridge inside the student's own dev server. (c) WebAssembly Postgres in the browser.

Chose (b). (a) is a security and privacy problem — credentials leaving the machine, a service to operate, and a target worth attacking, all for a classroom tool. (c) can't reach a database that lives on localhost anyway. (b) keeps credentials on the student's machine, needs no infrastructure, and costs ~160 lines.

Cost. The feature only exists when running locally. Handled explicitly: the static build detects the missing endpoint and explains why, rather than failing mysteriously — the whole app stays useful on GitHub Pages, minus one feature.

Defense in depth. `SET SESSION CHARACTERISTICS AS TRANSACTION READ ONLY`, only catalog + `SELECT … LIMIT` queries, escaped identifiers, hard row caps, no server-side persistence.

III.9 A type pass just for integer division

Problem. Postgres computes `7 / 2 = 3`, and the course teaches this gotcha. JS has no integer type, so a naive evaluator returns 3.5 — teaching the wrong lesson.

Options. (a) Ignore it. (b) Full static type checking of expressions. (c) A minimal `isIntExpr` predicate propagating int-ness where it matters.

Chose (c). Column metadata carries `isInt`; the predicate propagates through literals, arithmetic, `count/sum/min/max`, CASE branches, EXTRACT, and casts; division truncates only when both sides are integer-typed. Memoized per AST node so it costs nothing in loops.

Cost. It's a *partial* type system — it knows int vs. not-int and nothing else. It can be wrong in corners a real type checker would get right (e.g. deeply nested CASE branches mixing types).

Why it's still right. Reproducing the single most-taught SQL arithmetic gotcha is worth ~40 lines. Building a full type checker for a teaching tool would not be. Knowing the difference is the engineering judgment.

III.10 Immutable `Step[]` instead of incremental UI state

Chose: run the whole query eagerly, produce an immutable array of steps, and let the UI hold a single index.

Consequence. Stepping backward is free — no inverse operations, no recomputation, no possibility of forward/backward divergence. Play, scrub, and jump-to-chip are all the same operation. The stale-run detection is a string compare. Testing is trivial: assert on the array.

Cost. Peak memory holds every snapshot (bounded by the 100-row display cap, so it's small), and a query must finish before *any* step is visible — no streaming. At the enforced work ceiling that's imperceptible.

What I'd revisit. If the engine ever supported very large inputs, snapshots would need to become lazy (store row *ids* per step and reconstruct on demand). Deliberately not built — it's speculative complexity at

the current scale.

Part IV — Complexity & performance

IV.1 Hot paths

n = left/input rows, m = right rows, k = matches, g = groups, p = partition size.

Operation	Complexity	Note
Tokenize / parse	O(chars)	single pass, no backtracking
FROM (table instantiation)	O(n)	fresh column ids + row ids
Hash join (equi)	O(n + m + k)	build side m, probe side n
Nested-loop join (non-equi)	O(n × m)	guarded at 1.5M pairs
WHERE / HAVING	O(n × cost(predicate))	one pass, per-row snapshot
GROUP BY	O(n)	hash on `groupKey`, insertion-ordered
Aggregates	O(n) total	computed per group, memoized per group
DISTINCT / UNION dedupe	O(n)	hashed `groupKey` set
`IN (subquery)`	O(m) build + O(1) per probe	hashed set, cached if uncorrelated
Correlated subquery	O(n × subquery)	inherent — it re-runs per outer row
ORDER BY	O(n log n)	stable decorate-sort-undecorate
Window functions	O(n) partition + O(p log p) sort	per partition
Window frame (explicit ROWS)	O(n × frame)	naive per-row frame recompute

The one honest inefficiency: explicit `ROWS BETWEEN` frames recompute the aggregate over each row's frame rather than maintaining a running accumulator. That's O(n × frame width) where a sliding-window accumulator would be O(n). Fine at teaching scale, and I know it — a good thing to name in an interview *before* being asked.

IV.2 The join rewrite, measured

	Before (nested loop)	After (hash path)
Full `accounts ⋈ orders` (352 × 6,912)	2.4M pair evaluations — refused by guard	**38 ms**, all 8 steps
Max usable table size	~300 rows sampled	50,000 rows/table
Result fidelity	sample-dependent	cell-identical to `psql`

The hash path wins because it replaces "test every pair" with "bucket the right side once, then look up." Bucket insertion order preserves table order, so the output row order matches the nested-loop result exactly — which is why the differential test passes on both paths.

IV.3 Guard rails (and why each number)

Guard	Value	Rationale
Work meter	30M ticks	~a second of evaluation; aborts runaway queries with a teaching hint instead of freezing the tab
Nested-loop pairs	1.5M	only reachable without equi-keys; the message suggests adding an ON equality
Join result rows	500k	catches a wrong join key producing a cartesian explosion
Bridge rows/table	50,000	keeps a snapshot in memory and the transfer quick
Displayed rows/step	100	the animation is the lesson; more rows teach nothing
Displayed rows/result	1,000	the result is data, not animation — but still bounded
Animated rows	≤150	above this, FLIP animation costs CPU for no pedagogy

Design principle behind all of them: **fail with an explanation, never with a frozen tab.** Every guard's message names the likely cause and the fix.

Part V — Trust: testing & CI

18. Verification

Four layers, shallowest to deepest.

18.1 The layers

- `npm test` — **62 Vitest unit tests**, no database required, covering what the differential test structurally cannot see:

Suite	Tests	Covers
`errors.test.ts`	18	every classic mistake pinned to message, hint, and source span
`steps.test.ts`	13	phase order, kept/dropped marks, the Mattel ghost row, id stability across ORDER BY, timeline source names and color stability
`csv.test.ts`	11	quoting, CRLF, embedded newlines, type inference, ragged rows, name collisions
`values.test.ts`	10	three-valued logic, interval decomposition, NULL-aware grouping keys
`engine.test.ts`	10	golden Postgres semantics that need no DB (integer division, `NOT IN` with NULL, peer-inclusive running totals, empty-input aggregates)

- `npm run smoke` — 26 engine cases (foundations through capstones, including **every curated example**) printing step traces and results. A broken example fails CI.
- `npm run verify` — the **differential test**: creates a scratch Postgres database (`eidossql_verify`), loads the exact embedded dataset, runs a **61-query battery** through both the engine and Postgres, and diffs results cell-by-cell (NULL-tokenized, float-tolerant, multiset comparison unless the query has a determining ORDER BY), then drops the database. Coverage: every join type on both planner paths, grouping edge cases, all window function classes and frames, set ops, correlated and uncorrelated subqueries, date/interval math, casts, integer division, and the three capstones. **Current status: 61/61 identical.**
- **Continuous integration** (`.github/workflows/ci.yml`): every push runs typecheck → oxlint → unit suites → the full differential test against a real `postgres:16` **service container** → production build. The oracle isn't mocked in CI: the pipeline boots an actual PostgreSQL and diffs the engine against it. A second workflow (`deploy.yml`) publishes the static build to GitHub Pages on main.

18.2 War stories — three bugs, three lessons

① **Aggregates over zero rows.** `SELECT count(*) FROM orders WHERE id < 0` returned *no rows*. Postgres returns **one** row containing `0`. The rule: an aggregate with no GROUP BY always produces

exactly one row, even over an empty input — because the whole table is one implicit group, and an empty group is still a group. I didn't know this rule; the differential test did. Fix: seed a single empty group when there's no GROUP BY and no rows. *Lesson: a reference implementation encodes rules you don't know you don't know.*

② **Window functions over grouped rows.** `rank() over (order by sum(x))` with a GROUP BY failed its value lookup. The cause was subtle: window values are keyed by row id, but the group-context constructor was handing back the *first member row* of each group rather than the group row itself, so the lookup missed. Fix: pass the group row (whose id is `g:<key>`) as the context row. *Lesson: identity schemes must be consistent everywhere, and a mixed-feature query is where inconsistency surfaces.*

③ **The CSV blank-line question (found by writing a test).** Writing `csv.test.ts` forced a question the implementation had answered by accident: is a blank line a NULL row, or noise? I'd written a filter that made single-column files behave differently from multi-column ones — a silent inconsistency. Decided explicitly (skip blank *lines*, pandas-style; blank *cells* become NULL), implemented it uniformly, and pinned both behaviors with tests. *Lesson: writing tests is a design review — it surfaces decisions you made without noticing.*

18.3 Why this pyramid, in one sentence each

- **Unit tests** protect the parts with *no reference implementation* — error messages, step structure, the visual grammar.
- **The differential test** protects the parts where a reference exists, and is stronger than any expectation I could write by hand.
- **CI** makes both claims true continuously instead of "true on my machine the last time I remembered to run it."

Part VI — Limits, workflows, and reflection

19. Known limitations & deviations from Postgres

Statements: SELECT only — no INSERT/UPDATE/DELETE/DDL, one statement at a time. Within SELECT, not supported: `ANY` / `ALL` / `SOME` comparisons, `NATURAL JOIN`, `LATERAL`, named windows (`WINDOW w AS …`), `GROUPING SETS/ROLLUP/CUBE`, `FILTER (WHERE …)`, `WITHIN GROUP`/ordered-set aggregates, `string_agg` / `array_agg`, arrays/JSON operators, `RANGE` / `GROUPS` explicit frames (`ROWS` frames and defaults are supported), recursive CTEs, `TABLESAMPLE`, collation controls.

Deliberate deviations: `USING` keeps both join columns instead of merging (§11); `to_char` implements the common patterns only; very lenient number-vs-numeric-string comparisons (teaching kindness over strictness); row output order without ORDER BY follows the engine's deterministic pipeline order (Postgres promises nothing there anyway).

Practical ceilings: see §IV.3.

Being able to recite this list is itself an interview asset — it shows the scope was *chosen*, not accidental.

20. Workflows

```
npm run dev      # dev server + Postgres bridge  → http://localhost:5173
npm run build    # static production build (dist/) — bridge not included
npm run preview  # serve the build locally (bridge included)
npm test         # 62 Vitest unit tests (no database needed)
npm run lint     # oxlint
npm run smoke    # engine smoke test (26 cases)
npm run verify   # differential test vs local Postgres (needs psql/createdb)
```

Environment note: if `npm run dev` or `npm run lint` fails with "Cannot find native binding" (a known npm optional-dependency bug), run `npm install --no-save @rolldown/binding-darwin-arm64@<version>` (or `@oxlint/binding-darwin-arm64` for the linter).

21. Demo script (5 minutes, for a person)

1. **Open on the HAVING example.** "This is a join, a group-by, and a filter on the group." Press Play. Let the eight steps run once without talking.

2. **Step back to the JOIN.** Point at the red ✕ row: "Mattel has no orders. An inner join deletes it. That's the whole difference between inner and left join, and you can see it."
3. **Step to GROUP BY.** "Thirty rows become eleven groups — same colors, contiguous." Then the collapse: "each group becomes exactly one row, and the aggregate is computed across the rows it swallowed."
4. **Step to SELECT.** "Notice this is step six of eight. SELECT is almost the *last* thing that happens — that's why you can't use a SELECT alias in WHERE."
5. **Load the capstone.** Point at the timeline: "colors tell you which table each step is about. This chip is half-and-half — it's the join. These green ones are inside the CTE, and *this* green one is the subquery in the HAVING clause reading that same CTE."
6. **Break something.** Type `where total_spent > 5000` (an alias in WHERE) and run: "and when you get it wrong, it explains the rule instead of saying 'column does not exist.'"

22. What I'd do differently, and what I learned

Differently:

1. **Write the differential harness first.** It's what made the engine trustworthy; earlier would have meant fewer bugs living longer.
2. **Separate evaluation from narration** (§III.2) — an event-emitting evaluator plus a narrator module would be more testable and reusable.
3. **Design the join planner up front.** Sampling was a workaround for an algorithm choice I hadn't examined; the planner was under a day's work once I actually looked (§III.4).
4. **Decide edge-case semantics explicitly**, rather than discovering (as with CSV blank lines) that the implementation had decided for me.

Learned:

- *A reference implementation is the best test oracle available.* Assert agreement, not expectations.
- *Know what your strongest test can't see.* The differential test is blind to errors and to steps — the two things students actually interact with.
- *A limitation you've rationalized is still a limitation.* "Sampling is pedagogically correct" was a story I told myself about an O(n×m) join.
- *Data-model invariants can carry UX requirements.* Stable row ids are an engine-level promise that makes a UI-level animation trivially correct.
- *Error messages are product surface.* They deserve the same design attention — and the same tests — as features.

Appendix A — Interview Q&A

A-1. Why not just use SQLite/WASM? It only exposes final results. The product needs the working set *between* clauses, with rows retaining identity so motion is meaningful. There's no hook for that, and intermediate states can't be reconstructed from a final result — you can't recover pre-GROUP BY rows from grouped output. See §III.1.

A-2. How does the engine actually work, end to end? Tokenizer → recursive-descent parser producing an AST where every node carries a source span → an executor that walks the AST in SQL's *logical* clause order, evaluating for real and pushing an immutable snapshot after each stage. The UI receives `Step[]` and renders index N.

A-3. Why is a source span on every AST node important? One invariant, three features: the editor highlights the clause belonging to the current step, errors underline the exact offending token, and each step knows which text produced it. It's nearly free during parsing and impossible to retrofit reliably afterward.

A-4. How do you make the animation correct rather than decorative? The engine guarantees stable row ids across snapshots (`left⋈right` , `left⋈ø` , `g:<key>`). React keys off those ids; framer-motion FLIPs the layout delta. The engine contains no animation code and the UI contains no SQL logic — the id contract is the whole interface.

A-5. Walk me through the join implementation. Flatten ON into AND-conjuncts; classify each `=` by which side its columns come from; clean left/right splits become hash keys, everything else becomes a residual predicate. Build a hash map on the right side keyed by `groupKey` , excluding NULL keys (NULL never equals anything); probe from the left and apply residuals to candidates. No equi-key at all ⇒ nested loop, guarded. Outer joins then append the unmatched rows as NULL-extended.

A-6. Why exclude NULL keys from the hash build? Because `NULL = NULL` is unknown, not true, in SQL. A NULL-keyed row can never satisfy an equality, so it must never land in a bucket where it could be probed into a match. It still participates as an unmatched row in outer joins.

A-7. How do you know a subquery is correlated? At runtime: the enclosing row context is pushed on an outer-scope stack with a boolean flag; if name resolution falls through to that frame, the flag trips. Correlated ⇒ re-run per row, don't cache. It reuses the resolver that has to exist anyway and drives the user-facing explanation too. §III.5.

A-8. What's the time complexity of the pipeline? Parse O(chars); hash join O(n + m + k); GROUP BY O(n) via hashing; ORDER BY O(n log n); windows O(n) partitioning plus O(p log p) sorting per partition. The known inefficiency is explicit `ROWS` frames, recomputed per row rather than via a sliding accumulator. See Part IV.

A-9. How did you test something with this much semantic surface? Differential testing as the primary oracle — 61 queries through both my engine and a real Postgres, diffed cell by cell, running in CI against a service container. Plus 62 unit tests aimed at what that oracle can't see: error messages, step structure, and value-level semantics. §III.7, §18.

A-10. Tell me about a bug your tests caught. Three, in §18.2. The best one: aggregates over zero rows must return one row, not none — a SQL rule I didn't know, which only a reference implementation would have told me.

A-11. What's the weakest part of the codebase? `executor.ts` at ~2,000 lines mixes evaluation with narration. It's cohesive (one pipeline, one order) but it's the file I'd split first — evaluator emits events, narrator builds steps. §III.2.

A-12. How does connecting to a real database work in a browser? It doesn't — browsers can't speak the Postgres wire protocol. A ~160-line Vite middleware runs inside the student's own dev server, introspects the schema, and returns a snapshot. Read-only session, capped rows, credentials never leave the machine. On a static deploy the endpoint is absent and the UI explains why instead of erroring. §14, §III.8.

A-13. Security considerations? The bridge only ever runs locally and only issues catalog queries and `SELECT … LIMIT`; the session is forced read-only; identifiers are escaped; nothing is persisted server-side. CSVs are parsed in-browser and never uploaded. The static build has no server component at all.

A-14. How do you handle NULLs? Three-valued logic at the comparison layer: comparisons return `null` when either side is NULL, `truthy()` treats NULL as "reject," and AND/OR/NOT propagate unknowns. But `groupKey` deliberately treats NULLs as *equal*, because GROUP BY/DISTINCT/set-ops do. The two rules coexisting is exactly why `NOT IN (… NULL)` matches nothing — which is a tested case.

A-15. Why TypeScript, and did the types earn their keep? Yes, twice concretely: adding `sources` to the `Relation` type turned "find every place a relation is constructed" into a compiler task — it listed all 16 sites; and the discriminated-union AST makes the executor's switch statements exhaustively checked, so adding a node type surfaces every place that must handle it.

A-16. How would you scale this to a million rows? I wouldn't animate it — but I'd separate computation from presentation: stream the pipeline with iterators, store per-step row *ids* rather than materialized snapshots, and reconstruct display rows lazily. The current design deliberately trades that for simplicity; the ceilings are enforced and explained rather than hidden (§IV.3).

A-17. What would you add next? Follow-a-row mode (click a row, spotlight it across every step — nearly free given stable ids), shareable URLs encoding query + dataset + step index, and a predict-mode quiz that hides a step's result until the student commits to an answer.

A-18. How is this different from `EXPLAIN`? `EXPLAIN` shows the *physical* plan — the strategy the optimizer chose. EidosSQL shows the *logical* semantics with actual data: what rows exist after each clause. Complementary: one answers "how will the DB execute this," the other "what does this query mean."

A-19. Who is it for, and did it work? Built for a specific user — the professor who hired me as a TA, and her students, who kept asking her to redraw tables on a whiteboard. That constraint shaped features that a generic tool wouldn't have: presentation mode for the projector, a print stylesheet for handouts, and error messages written the way a TA would explain them.

A-20. What does the CI actually run? Typecheck (both tsconfigs), oxlint, 62 unit tests, the 61-query differential suite against a `postgres:16` service container, and a production build — on every push. A separate workflow deploys the static build to Pages.

Appendix B — Concept glossary

The vocabulary this project uses. Being able to name these precisely is worth as much in an interview as having implemented them.

Recursive-descent parser — a top-down parser with one function per grammar production; operator precedence is encoded in the order functions call each other. *Here:* `parser.ts`, `parseOr → … → parsePrimary`.

AST (abstract syntax tree) — the tree-shaped representation of parsed source. *Here:* `ast.ts`, where every node also carries `{start, end}`.

Source span — a `{start, end}` character range tying a node back to the text that produced it. *Here:* powers the clause spotlight and error underlines.

Logical clause order — the order SQL *means* its clauses (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY), as opposed to the order they're written. *Here:* the executor's pipeline order, and the app's core lesson.

Relation — in relational algebra, a set of tuples with a schema; a "table-valued intermediate result." *Here:* `{cols, rows, sources}`, threaded through every operator.

Hash join — join by building a hash table on one input's keys and probing with the other: O(n + m) instead of O(n × m). *Here:* §11.

Nested-loop join — compare every pair; the only option without an equality predicate. *Here:* the fallback path, guarded.

Equi-join key vs. residual predicate — the part of an ON clause that is a clean `left = right` equality (usable as a hash key) versus everything else (must be tested per candidate pair). *Here:* the planner's classification.

Three-valued logic (3VL) — SQL's TRUE/FALSE/UNKNOWN, where any comparison with NULL yields UNKNOWN and UNKNOWN rejects rows. *Here:* `compareValues` returning `null`, `truthy()`.

Correlated subquery — a subquery referencing a column from the enclosing query, so it must be re-evaluated per outer row. *Here:* detected by the outer-scope tripwire.

Window function — computes over a set of related rows ("the window") *without* collapsing them, unlike aggregates with GROUP BY. *Here:* `row_number/rank/lag/…` plus aggregates with OVER.

Window frame — which rows within a partition a window function sees; the default with ORDER BY includes peers (ties), which is why running totals tie. *Here:* §10's window section.

Peer rows — rows tied on the ORDER BY key within a partition; they share a frame under the default RANGE semantics.

CTE (common table expression) — a named `WITH` subquery usable like a table. *Here:* executes first as a nested step group, then registers in the environment.

Projection / selection — relational algebra: projection chooses columns (SQL's SELECT list), selection chooses rows (SQL's WHERE). *Here:* two distinct pipeline stages, deliberately far apart in the ordering.

Cartesian product / cross join — every pairing of two inputs; what you get when a join condition is missing. *Here:* explained in the missing-ON error.

FLIP animation — "First, Last, Invert, Play": measure an element before and after a layout change, then animate the delta. Requires stable identity. *Here:* framer-motion `layout` on rows keyed by engine-assigned ids.

Differential testing — validating an implementation by comparing its output against a reference implementation on the same inputs, rather than against hand-written expectations. *Here:* `scripts/verify.ts` vs. Postgres.

Test oracle — whatever decides if output is correct. *Here:* Postgres for results; hand-written assertions for errors and steps.

Service container — a dependency (here, `postgres:16`) that CI runs alongside the job so integration tests use the real thing rather than a mock.

Graceful degradation — a feature that can't work in an environment disables itself with an explanation instead of failing. *Here:* the Postgres bridge on a static deploy.

Memoization — caching a pure function's result per input. *Here:* `isIntExpr` per AST node; aggregate values per group.

Idempotent/immutable state — the run is an immutable `Step[]`; the UI holds an index. Stepping backward needs no inverse operation.

Appendix C — Crown-jewel code

Four excerpts worth being able to explain line by line. Lightly trimmed; `…` marks omissions.

C-1. The join planner's side classification (`executor.ts`)

Decides whether an ON conjunct can become a hash key. `bail` is the important part: anything the planner can't reason about cleanly (subqueries, aggregates, ambiguous or unknown columns) falls back to the safe path rather than guessing.

```
const sideOf = (e: Expr): 'left' | 'right' | 'both' | 'none' | 'bail' => {
  let inL = false, inR = false, bail = false;
  this.walk(e, (x) => {
    if (x.kind === 'subquery' || x.kind === 'exists' || (x.kind === 'in' && x.query)) bail =
true;
    if (x.kind === 'call' && (x.over || AGG_FUNC_NAMES.has(x.name))) bail = true;
    if (x.kind === 'col') {
      let l = false, r = false;
      try { l = this.findColIndex(x, left.cols) !== null; } catch { bail = true; }
      try { r = this.findColIndex(x, right.cols) !== null; } catch { bail = true; }
      if (l && r) bail = true;          // ambiguous — let normal evaluation report it
      else if (l) inL = true;
      else if (r) inR = true;
      else bail = true;                 // unknown column — normal evaluation raises it
    }
  });
  if (bail) return 'bail';
  if (inL && inR) return 'both';
  …
};
```

Then: a conjunct whose sides split cleanly becomes `equiL[i] = equiR[i]`; everything else joins `residual[]` and is tested per candidate pair.

C-2. Correlation detection by tripwire (`executor.ts`)

```
evalSubqueryRel(q: Query, ctx: Ctx, label: string): Relation {
  const cached = this.subqCache.get(q);
  if (cached) return cached;

  const used = { hit: false };            // ← the tripwire
  this.outerStack.push({ ctx, used });
  try {
    rel = this.execQuery(q, ctx.env, [...ctx.path, label]);
  } finally {
    this.outerStack.pop();
  }
  …
  // narration branches on the same flag that decides caching
  desc: used.hit
    ? 'This subquery uses columns from the outer query (it is *correlated*), so it re-runs for
every outer row…'
    : 'This subquery does not depend on the outer row, so SQL evaluates it once and reuses the
value. '
  …
  if (!used.hit) this.subqCache.set(q, rel);   // only uncorrelated results are cacheable
  return rel;
}
```

`used.hit` is set inside column resolution, when a lookup misses every local scope and finds the name in an outer frame. One boolean answers "is this correlated," decides caching, and writes the explanation.

C-3. Stable row identity through a join (`executor.ts`)

```
const row = { id: `${l.id}⋈${r.id}`, vals: [...l.vals, ...r.vals] };   // matched pair
…
const ghost = { id: `${l.id}⋈ø`, vals: [...l.vals, ...nullsR] };        // no match
if (type === 'left' || type === 'full') {
  resultRows.push(ghost);
  rowStatus.set(ghost.id, 'new');
  notes.set(ghost.id, 'no match — kept, filled with NULLs');
} else if (type === 'inner') {
  rowStatus.set(ghost.id, 'dropped');                       // shown, then removed
  notes.set(ghost.id, 'no match — removed by INNER JOIN');
}
```

Note that the inner-join case still *creates* the ghost row — it's pushed to the visualization but not to the result. That's the "show what was deleted" teaching move, and it's why inner vs. outer join is visible rather than described.

C-4. The differential comparison (`scripts/verify.ts`)

```
function cellsEqual(a: string, b: string): boolean {
  if (a === b) return true;
  const na = Number(a), nb = Number(b);
  if (!isNaN(na) && !isNaN(nb) && a.trim() !== '' && b.trim() !== '') {
    return Math.abs(na - nb) <= 1e-6 * Math.max(1, Math.abs(na), Math.abs(nb));
  }
  return false;
}
```

Three deliberate choices: NULLs are tokenized (`<<NULL>>`) so they compare as values rather than empty strings; floats compare with relative tolerance (both sides do IEEE arithmetic in different orders); and rows are compared as a **multiset** unless the query has a top-level ORDER BY — asserting an order SQL doesn't guarantee would produce false failures.

Appendix D — Project metrics

Metric	Value
Total TypeScript (src + server)	~5,460 lines
SQL engine	3,759 lines (executor 2,011 · parser 878 · functions 319 · ast 215 · tokens 148 · values 109 · steps 72)
React UI	1,177 lines · CSS 1,090 lines
Runtime dependencies	4 declared; the **browser bundle** uses 3 (react, react-dom, framer-motion). `pg` is used only by the dev-server bridge. The **SQL engine has zero** — it's pure TypeScript
SQL keywords tokenized	60
Scalar functions	35, plus 5 aggregates and 8 window functions
Error sites / with teaching hints	64 / 37
Curated examples	21, across 6 teaching groups
Unit tests	62 (5 suites)
Differential queries vs. Postgres	61 — **61/61 identical**
Smoke cases	26 (includes every example)
CI stages	typecheck → lint → unit → differential (postgres:16 service) → build
Data sources	3 (embedded, live Postgres, CSV) normalized to one `Dataset` shape

Guide last updated for v1.1 — July 2026.